



Práctica 3 – Algoritmos sobre grafos

Los objetivos de esta práctica son:

- practicar resolver problemas sobre grafos con algoritmos, y
- familiarizarse con algoritmos de búsqueda comunes sobre grafos (BFS y DFS).

Los ejercicios marcados con el símbolo ★ constituyen un subconjunto mínimo de ejercitación. Sin embargo, aconsejamos fuertemente hacer todos los ejercicios.

Los ejercicios marcados con el símbolo ⓘ tienen una ayuda al final de la guía.

Ejercicio 1 (*Representación de grafos*) ★

Para un grafo G , el *conjunto de vecindarios* (o conjunto de *adyacencias*) es un par (V, N) donde N es una función que asigna a cada vértice $v \in V(G)$ su correspondiente conjunto de vértices adyacentes $N(v)$, es decir, su vecindario.

En todos los grafos G que tengamos como entrada los vértices van a ser el conjunto $\{0, \dots, |V(G)| - 1\}$.

Discutir (brevemente) las ventajas y desventajas en cuanto a la complejidad temporal y espacial de las siguientes implementaciones de un conjunto de vecindarios para un grafo G , de acuerdo a las siguientes operaciones:

Operaciones

1. Inicializar la estructura a partir de una secuencia de vértices y una secuencia de aristas de G .
2. Determinar si dos vértices v y w son adyacentes.
3. Recorrer y/o procesar el vecindario $N(v)$ de un vértice v dado.
4. Insertar un vértice v con su conjunto de vecinos $N(v)$.
5. Insertar una arista (v, w) .
6. Remover un vértice v con todas sus adyacencias.
7. Remover una arista (v, w) .
8. Mantener un orden de $N(v)$ de acuerdo a algún invariante que permita recorrer cada vecindario en un orden dado.

Estructuras de datos

1. La función N se representa con una secuencia (arreglo dinámico o lista enlazada) que en cada posición v tiene el conjunto $N(v)$ implementado a su vez sobre una secuencia (arreglo dinámico o lista enlazada). Cada vértice es una estructura que tiene un índice para acceder en $O(1)$ a $N(v)$. Esta representación se conoce comúnmente como *lista de adyacencias*.
2. Ídem anterior, pero cada $w \in N(v)$ se almacena junto con un índice a la posición que ocupa v en $N(w)$. Esta representación también se conoce como *lista de adyacencias*, pero tiene información para implementar operaciones dinámicas.
3. $N(v)$ se representa con un arreglo dinámico que en cada posición i tiene un arreglo dinámico de booleanos A_i con $|V(G)|$ posiciones tal que $A_i[j]$ es verdadero si y solo si i es adyacente a j . Esta representación se conoce comúnmente como *matriz de adyacencias*.



4. $N(v)$ se representa con un arreglo dinámico que en cada posición tiene el conjunto $N(v)$ implementado con una tabla de hash. Esta representación es un mix entre las representaciones clásicas de matriz de adyacencias y lista de adyacencias.

Ejercicio 2 (Triángulos) ★

Un *triángulo* de un grafo G es una tripla $\{v, w, z\}$ que induce un subgrafo completo (de tamaño 3; ver Figura 2). Considerar los siguientes algoritmos para decidir si un grafo G de n vértices y $m > n$ aristas tiene un triángulo.

Algoritmo cúbico

Computar la matriz de adyacencias A de G .

Retornar verdadero si existen $v, w, z \in V(G)$ tales que $A_{vw}A_{wz}A_{vz} = 1$.

Algoritmo cuadrático

Computar las listas de adyacencias N de G .

Para cada $v \in V(G)$:

 Marcar cada $w \in N(v)$.

 Retornar verdadero si existe $wz \in E(G)$ tal que w y z están marcados.

 Desmarcar cada $w \in N(v)$.

- Argumentar por qué cada algoritmo es correcto.
- Demostrar que el algoritmo cúbico requiere tiempo $\Theta(n^3)$ y el algoritmo cuadrático requiere tiempo $\Theta(nm) = O(m^2)$. ⓘ
- Determinar un mejor y un peor caso para cada uno de los algoritmos.
- Implementar los dos algoritmos en su lenguaje de programación favorito, y probarlos con muchos grafos de diferentes tamaños y densidades. Consideren hacer que los grafos se generen automáticamente, para poder comparar fielmente los algoritmos. Comparar los tiempos de ejecución con las complejidades teóricas.

Ejercicio 3 (Orden topológico) ★

Dado un grafo dirigido D , un *orden topológico* de D es un orden parcial $<$ de los vértices $V(D)$ tal que si $v \rightarrow w \in E(D)$ entonces $v < w$.

- Demostrar que si un digrafo no tiene ciclos, entonces tiene por lo menos un vértice con grado de entrada igual a 0.
- Usando el Item a), describir un algoritmo para construir un orden topológico en un digrafo sin ciclos. Demostrar su correctitud y determinar su complejidad temporal y espacial. No vale usar DFS.
- Demostrar que un digrafo admite un orden topológico si y solo si no tiene ciclos, es decir, es un DAG (digrafo acíclico). Demostrar la ida por contrarrecíproco.
- Si no lo hicieron todavía, mejorar el algoritmo del Item b) para que corra en tiempo $O(|V(D)| + |E(D)|)$. ⓘ

Ejercicio 4 (Ciclos rho) ★

Decimos que un digrafo (con *loops*) tiene forma de ρ cuando todos sus vértices tienen grado de salida igual a 1 (Figura 1).

- Demostrar en forma constructiva que si un digrafo es conexo y tiene forma de ρ entonces tiene un único ciclo dirigido. Notar que si $v \rightarrow v$ es un *loop*, entonces v, v es un ciclo. Recordar que un digrafo es conexo cuando su grafo subyacente es conexo. ⓘ

- b) Diseñar un algoritmo para encontrar todos los ciclos de un digrafo con forma de ρ (no necesariamente conexo).

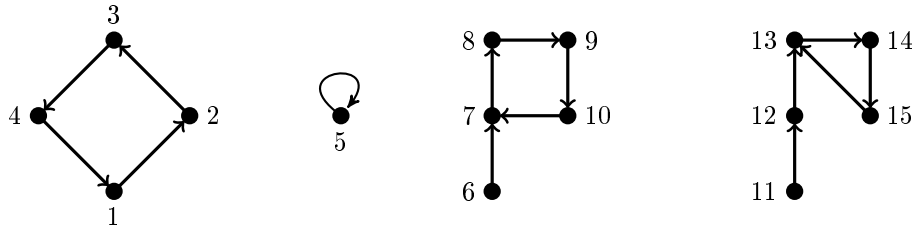


Figura 1: Un digrafo disconexo con forma de ρ ; cada componente conexa tiene forma de ρ .

Ejercicio 5 (Gemelos y mellizos)

Recordar que el *vecindario* de un vértice v es el conjunto $N(v)$ que contiene a todos los vértices adyacentes a v . El *vecindario cerrado* es $N[v] = N(v) \cup \{v\}$. Dos vértices u y v son *mellizos* cuando $N(u) = N(v)$, mientras que son *gemelos* cuando $N[u] = N[v]$ (Figura 2).

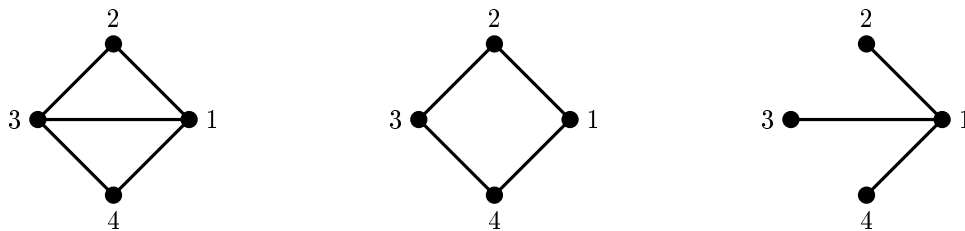


Figura 2: El grafo diamante (izquierda), el grafo C_4 (centro) y el grafo garra (derecha). Los vértices 1 y 3 son gemelos en el diamante porque $N[1] = N[3] = \{1, 2, 3, 4\}$ mientras que 2 y 4 son mellizos porque $N(2) = N(4) = \{1, 3\}$. En la garra, $N(2) = N(3) = N(4) = \{1\}$ y, por lo tanto, 2, 3 y 4 son mellizos, mientras que en C_4 la partición en mellizos es $\{2, 4\}$ y $\{1, 3\}$. El diamante tiene 2 triángulos $\{1, 2, 3\}$ y $\{1, 3, 4\}$ mientras que C_4 y la garra no tienen triángulos. Notar que el diamante es threshold porque $N(2) = N(4)$ y $N[2] \subseteq N[1] = N[3]$; ciertamente $(2, 4, 1, 3)$ es una descomposición threshold del diamante. En cambio, C_4 no es threshold porque tanto $N(1)$ y $N(2)$ como $N[1]$ y $N[2]$ son incomparables por inclusión. Finalmente, la garra es threshold (por qué?).

- Observar que las relaciones de mellizos y gemelos son relaciones de equivalencia (i.e., son reflexivas, transitivas y simétricas)¹.
- Probar que el Algoritmo 1 encuentra la partición de $V(G)$ en vértices gemelos.
- Describir la implementación del algoritmo, especificando las estructuras de datos utilizadas. La mejor implementación que conocemos tiene complejidad temporal $O(n + m)$.
- ¿Qué debería modificarse para que el algoritmo encuentre la partición en vértices mellizos?

¹Por si no recuerdan las definiciones de estas propiedades: https://es.wikipedia.org/wiki/Relaci%C3%B3n_de_equivalencia.



Algoritmo 1 Partición en vértices gemelos

```
 $\mathcal{P} \leftarrow \{V(G)\}$ 
Sea  $v_1, \dots, v_n$  un ordenamiento cualquiera de  $V(G)$ 
for  $i \leftarrow 1$  to  $n$  do
     $\mathcal{P}' \leftarrow \emptyset$ 
    for  $W \in \mathcal{P}$  do
        if  $W \cap N[v_i] \neq \emptyset$  then
             $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{W \cap N[v_i]\}$ 
        if  $W \setminus N[v_i] \neq \emptyset$  then
             $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{W \setminus N[v_i]\}$ 
     $\mathcal{P} \leftarrow \mathcal{P}'$ 
return  $\mathcal{P}$ 
```

Ejercicio 6 (*Grafos threshold – Caracterización*)

Un grafo G es *threshold* si para cada par de vértices $u, v \in V(G)$ tales que $d(u) \leq d(v)$ ocurre que $N(u) \subseteq N(v)$ o $N[u] \subseteq N[v]$ (ver Figura 2).

- Demostrar que si G es un grafo threshold, entonces los vértices de grado k son todos mellizos entre sí, o todos gemelos entre sí, para todo $0 \leq k \leq n-1$.
- Demostrar que si G es un grafo threshold, entonces tiene algún vértice de grado 0 o alguno de grado $n-1$.
- Demostrar que si G es un grafo threshold, entonces $G \setminus \{v\}$ es un grafo threshold para todo $v \in V(G)$.
- Decimos que un grafo H tiene una descomposición threshold si $V(H)$ admite un ordenamiento $v_1, \dots, v_n$ tal que v_i es un vértice de grado 0 o de grado $i-1$ en el subgrafo de H inducido por $\{v_1, \dots, v_i\}$. Usando los Items b) y c), demostrar que G es un grafo threshold si y sólo si admite una descomposición threshold.
- Sea $v_1, \dots, v_n$ una descomposición threshold de G y $w_1, \dots, w_n$ una descomposición threshold de H . Demuestre que G es isomorfo a H si y solo si $f(v_i) = w_i$ es un isomorfismo (es una función biyectiva y mantiene las adyacencias). **i**

Ejercicio 7 (*Grafos threshold – Algoritmo*)

- Proponer una estructura de datos para un grafo threshold G basado en el Item d) del Ejercicio 6. La estructura de datos debe ocupar $O(n)$ bits. Indicar cómo se determina si dos vértices son adyacentes.
- Diseñar un algoritmo que, dado un grafo G cualquiera, determine si G es o no threshold. En caso afirmativo, el algoritmo debe devolver la representación del Item a) de este ejercicio.

Ejercicio 8 (*Aristas únicas*)

Dado un multigrafo D donde $V(D) = \{1, \dots, n\}$, queremos determinar aquellas aristas $v \rightarrow w$ de D tales que $w \rightarrow v$ no es arista de D .

- Describir un algoritmo de complejidad lineal que, dado un multigrafo G representado con un multiconjunto de aristas, determine las aristas (v, w) que no están repetidas en G .
- Describir un algoritmo lineal que, dado un multigrafo D representado con un multiconjunto de aristas, determine las aristas $v \rightarrow w$ tales que $w \rightarrow v$ no es arista de D .

Ejercicio 9 (*GeoCuadrados*)

Un camino P de v a w en un grafo G es *geodésico* cuando la cantidad de aristas de P es la mínima entre todos los caminos que unen a v con w (i.e., P es un camino mínimo entre v y w). El *cuadrado* de un grafo G es el



grafo G^2 que tiene los mismos vértices que G y donde vw son adyacentes si y solo si sus vecindarios cerrados tienen algún vértice en común.

- a) **(Difícil)** Demostrar que si G tiene un camino geodésico con al menos 4 aristas, entonces $\overline{G}^2$ es un grafo completo. **i**
- b) Demostrar, usando el inciso anterior y la técnica de reducción al absurdo, que si G tiene un camino geodésico con al menos 3 aristas, entonces $\overline{G}$ no tiene caminos geodésicos con más de 3 aristas.

Recorrido en profundidad (DFS)

Ejercicio 10 (Grafos bipartitos) ★

Sea T un árbol generador de un grafo (conexo) G con raíz r , y sean V y W los conjuntos de vértices que están a distancia par e impar de r , respectivamente.

- a) Demostrar que si existe una arista $(v, w) \in E(G) \setminus E(T)$ tal que $v, w \in V$ o $v, w \in W$, entonces el único ciclo de $T \cup \{(v, w)\}$ tiene longitud impar.
- b) Demostrar también que si toda arista de $E(G) \setminus E(T)$ une un vértice de V con otro de W , entonces (V, W) es una bipartición de G y, por lo tanto, G es bipartito.
- c) A partir de las observaciones anteriores, diseñar un algoritmo lineal para determinar si un grafo conexo G es bipartito. En caso afirmativo, el algoritmo debe retornar una bipartición de G . En caso negativo, el algoritmo debe retornar un ciclo impar de G . **Explicitar cómo es la implementación del algoritmo;** no es necesario incluir el código.
- d) Generalizar el algoritmo del inciso anterior a grafos no necesariamente conexos observando que un grafo G es bipartito si y solo si sus componentes conexas son bipartitas.
- e) Implementar el algoritmo del inciso anterior en su lenguaje de programación favorito y probarlo con un par de grafos.

Ejercicio 11 (Puentes vs. puntos de corte)

Sea G un grafo con al menos tres vértices. Un vértice $v \in V(G)$ es un *punto de corte* si $G \setminus v$ tiene más componentes conexas que G . Una arista $(v, w) \in E(G)$ es un *puente* si $G \setminus (v, w)$ tiene más componentes conexas que G .

Decidir si cada una de las siguientes afirmaciones es verdadera o falsa. Para las verdaderas, dar una demostración. Para las falsas, dar un contraejemplo.

- a) Si G es conexo y no tiene puentes, entonces G tiene exactamente un ciclo.
- b) Si G es conexo y tiene exactamente un ciclo, entonces G no tiene puentes.
- c) Si G es conexo y no tiene puntos de corte, entonces G no tiene puentes.
- d) Si G es conexo y no tiene puentes, entonces G no tiene puntos de corte.

Ejercicio 12 (Puentes) ★

Una arista de un grafo G es *puente* si su remoción aumenta la cantidad de componentes conexas de G . Sea T un árbol DFS de un grafo conexo G .

- a) Demostrar que (v, w) es un puente de G si y solo si (v, w) no pertenece a ningún ciclo de G .
- b) Demostrar que si $(v, w) \in E(G) \setminus E(T)$, entonces v es un ancestro de w en T o viceversa.

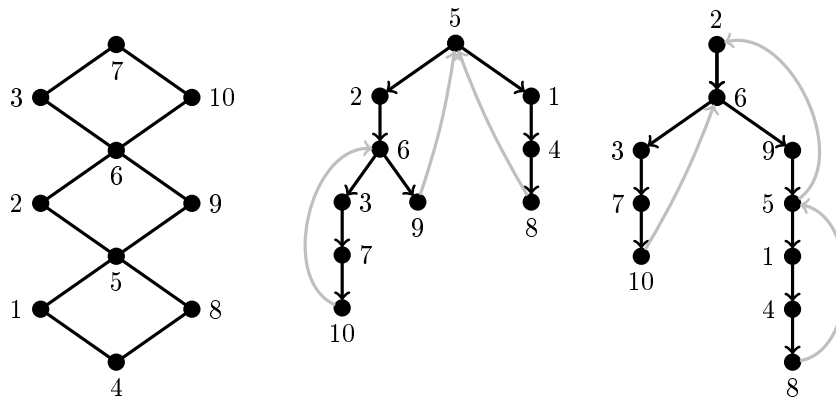


Figura 3: En el centro y la derecha se ven dos árboles DFS T_1 y T_2 del grafo G de la izquierda marcados en negro, junto con las aristas grises que completan $D(T_1)$ y $D(T_2)$.

- c) Sea $(v, w) \in E(G)$ una arista tal que el nivel de v en T es menor o igual al nivel de w en T . Demostrar que (v, w) es puente si y solo si v es el padre de w en T y ninguna arista de $G \setminus \{(v, w)\}$ une a un descendiente de w (o a w) con un ancestro de v (o con v).
- d) Dar un algoritmo lineal basado en DFS para encontrar todas las aristas puente de G . **i**

Ejercicio 13 (Orientaciones fuertes) ★

Una orientación de un grafo G es un grafo orientado D cuyo grafo subyacente es G . Para todo árbol DFS T de un grafo conexo G se define $D(T)$ como la orientación de G tal que $v \rightarrow w$ es una arista de $D(T)$ cuando v es el padre de w en T o w es un ancestro no padre de v en T (Figura 3).

- a) Observar que $D(T)$ está bien definido por el Ejercicio 12b).
- b) Demostrar que las siguientes afirmaciones son equivalentes:

- I) G admite una orientación que es fuertemente conexa.
- II) G no tiene puentes.
- III) Para todo árbol DFS T ocurre que $D(T)$ es fuertemente conexo.
- IV) Existe un árbol DFS T tal que $D(T)$ es fuertemente conexo.



- c) Dar un algoritmo lineal para encontrar una orientación fuertemente conexa de un grafo G cuando dicha orientación exista.

Ejercicio 14 (Orientación de calles)

La intendencia de una ciudad quiere orientar la mayor cantidad de calles posibles a fin de evitar accidentes. Actualmente, todas las calles son bidireccionales y unen un par de esquinas. Además, se puede alcanzar cualquier esquina de cualquier otra, y esto se desea mantener así. Modelar el problema de decidir qué calles se deben orientar y en qué sentido a fin de minimizar la cantidad de calles bidireccionales que quedan. Proponer un algoritmo de tiempo $O(n + m)$ para resolver el problema.

Recorrido en anchura (BFS)

Ejercicio 15 (Componentes conexas) ★



Escribir un programa en su lenguaje favorito que, dado un grafo G , determine en tiempo lineal la cantidad y el tamaño de sus componentes conexas. ¿Se puede hacer lo mismo con DFS? ⓘ

Ejercicio 16 (*Árboles geodésicos*) ★

Un árbol generador T de un grafo G es v -geodésico si la distancia entre v y w en T es igual a la distancia entre v y w en G para todo $w \in V(G)$. Demostrar que todo árbol BFS de G enraizado en v es v -geodésico. Dar un contraejemplo para la vuelta, i.e., mostrar un árbol generador v -geodésico de un grafo G que no pueda ser obtenido cuando BFS se ejecuta en G desde v .

Ejercicio 17 (*Ciclo mínimo par*)

Tenemos un grafo G cuyo ciclo más chico ya sabemos que es de tamaño par. Queremos determinar el tamaño del ciclo más chico de G .

- Supongamos que un vértice $v \in V(G)$ pertenece a un ciclo mínimo de G de tamaño $2k$. Demostrar que después de correr BFS desde el vértice v , existirá un vértice u visitado al menos dos veces que pertenecerá al nivel k del árbol BFS.
- Suponer que se corre BFS desde un vértice $v \in V(G)$. Demostrar que si no existe ningún ciclo de tamaño menor o igual a $2k$ en el grafo, entonces en los niveles menores o iguales a k no existe ningún vértice u que sea visitado dos veces.
- Concluir que G tiene un ciclo mínimo de tamaño $2k$ si y solo si existe un vértice $v \in V(G)$ tal que al correr BFS desde v , existe un vértice u en el nivel k que es visitado más de una vez.
- Diseñar un algoritmo de complejidad temporal $O(n \cdot m)$ que determine el tamaño del ciclo más chico de G , usando la caracterización del Item c).
- (**Difícil**) Mejorar el algoritmo a $O(n^2)$.

Ejercicio 18 (*Árbol geodésico de peso mínimo*)

Diseñar un algoritmo de tiempo $O(n + m)$ que, dado un grafo conexo G con pesos en sus aristas y un vértice v , determine el árbol de menor peso de entre todos los árboles v -geodésicos de G , es decir de los árboles v -geodésicos considerando el grafo sin pesos. El peso $w(T)$ de un árbol T es igual a la suma de los pesos de las aristas que pertenecen al árbol.

Demostrar que el algoritmo propuesto es correcto. ⓘ

Ayudas

Ejercicio 2

recordar que $O\left(\sum_{v \in V(G)} d(v)\right) = O(m)$.

Ejercicio 3

Mantener una cola con los vértices de grado de entrada 0. Una vez decidida la posición en el orden de un vértice, ¿qué vértices son afectados por esta decisión?

Ejercicio 4

notar que si se sacan los vértices con grado de entrada 0 en forma iterativa, entonces cada componente es un ciclo dirigido.

Ejercicio 5

demostrar por invariancia que, luego del paso i , u y w pertenecen al mismo conjunto de P_i si y sólo si $N[u] \cap \{v_1, \dots, v_i\} = N[w] \cap \{v_1, \dots, v_i\}$.

Ejercicio 6

Intentar hacer inducción en n .

Ejercicio 9

Siendo v y w los extremos de un camino geodésico de al menos 4 aristas en G , considerar los vecinos de v y w en $\underline{G}$.

Ejercicio 12

El algoritmo puede hacer un uso inteligente de un único DFS. Puede convertir separar el algoritmo en dos fases. La primera fase aplica DFS para calcular el mínimo nivel que se puede alcanzar desde cada vértice usando back edges que estén en su subárbol. La segunda fase recorre todas las aristas (sin DFS) para chequear la condición.

Ejercicio 13

Para **ii) $\Rightarrow$ iii)** observar que alcanza con mostrar que la raíz de $D(T)$ es alcanzable desde cualquier vértice v . Demuestre este hecho haciendo inducción en el nivel de v , aprovechando los resultados del Ejercicio 12.

Ejercicio 15

Es posible que tengan que correr BFS más de una vez.

Ejercicio 18

Pensar cuáles aristas pueden pertenecer a un árbol v -geodésico cualquiera, para elegir las que minimicen el peso total.